DIGITAL LOGIC DESIGN

(CE_118)

CHAPTER 3:

Register Transfer Specification & Design

(part_1)

Outline

- Register Transfer (RT) design:
  - FSMD model
  - RT specification with
    - Static-action tables
    - ASM charts
- Procedure for synthesis from RT specification
- Design Optimization through
  - Register sharing
  - Functional unit sharing
  - Bus sharing
  - Unit chaining and Multiclocking
- Design Pipelining
  - Unit pipelining
  - Control pipelining
  - Datapath pipelining
- Scheduling of flowcharts

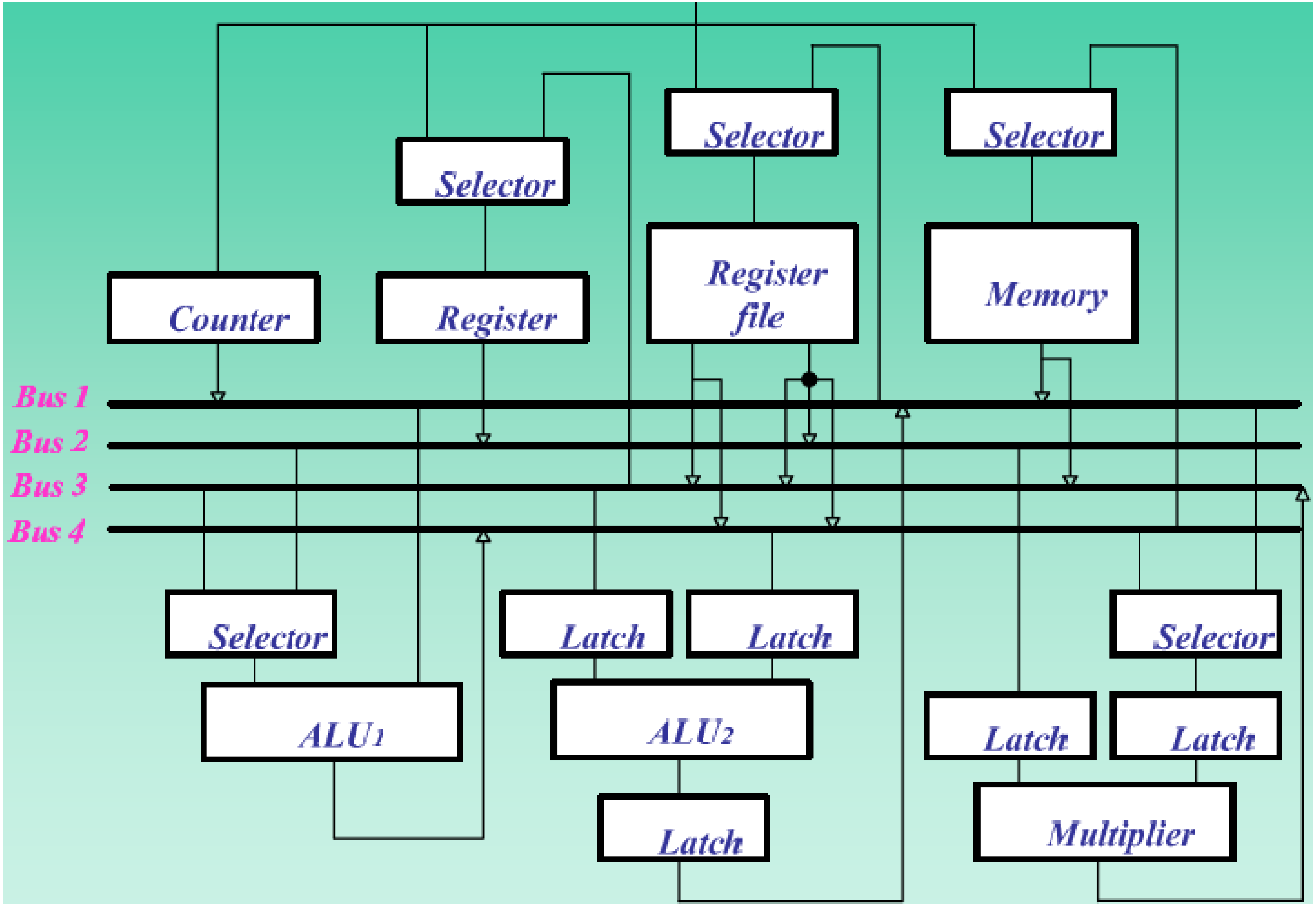
Outline

- Register Transfer (RT) design:
  - FSMD model
  - RT specification with
    - Static-action tables
    - ASM charts
- Procedure for synthesis from RT specification
- Design Optimization through
  - Register sharing
  - Functional unit sharing
  - Bus sharing
  - Unit chaining and Multiclocking
- Design Pipelining
  - Unit pipelining
  - Control pipelining
  - Datapath pipelining
- Scheduling of flowcharts

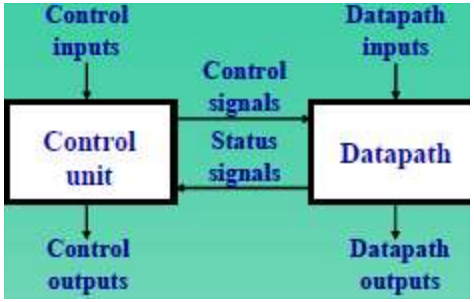
Register-transfer design

- Each standard or custom IC consists of one or more datapaths and control units.
- To synthesize such IC we introduce the model of a FSM with a datapath (FSMD).
- We demonstrate synthesis algorithms for FSMD model, including *component selection*, *resource sharing*, *pipelining and scheduling*.

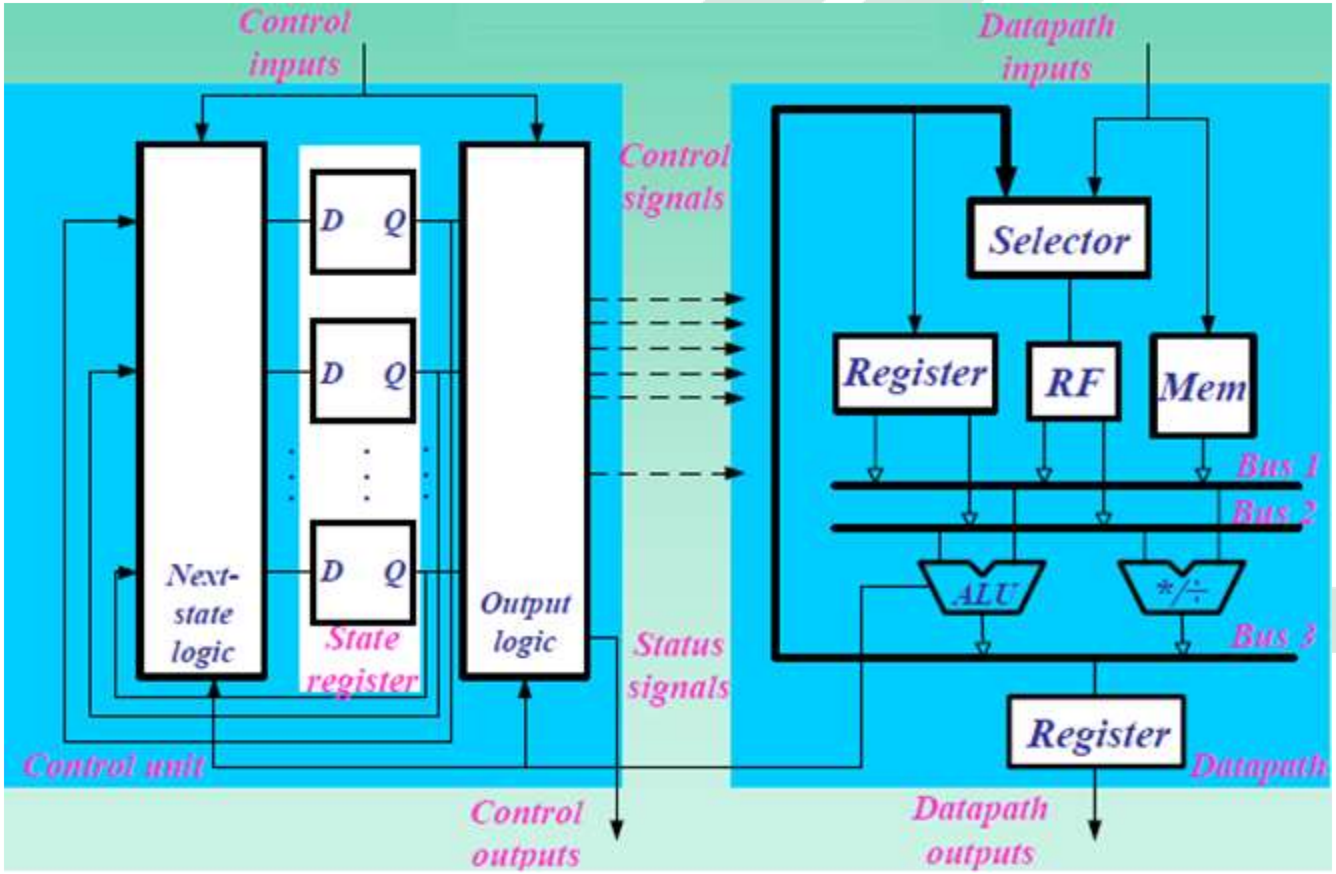
Example


Selector
Selector
Selector
Counter
Register
Register file
Memory
Bus 1
Bus 2
Bus 3
Bus 4
Selector
Latch
Latch
Selector
ALU1
ALU2
Latch
Latch
Latch
Multiplier

Design Model

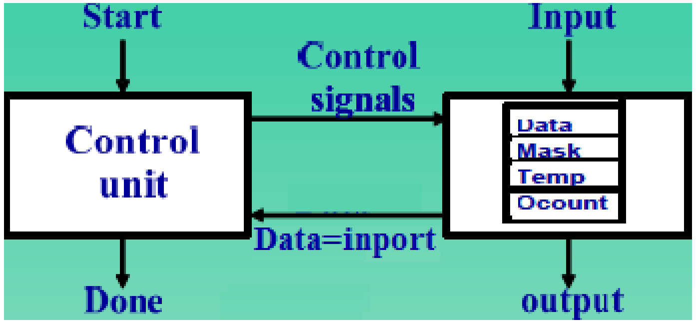


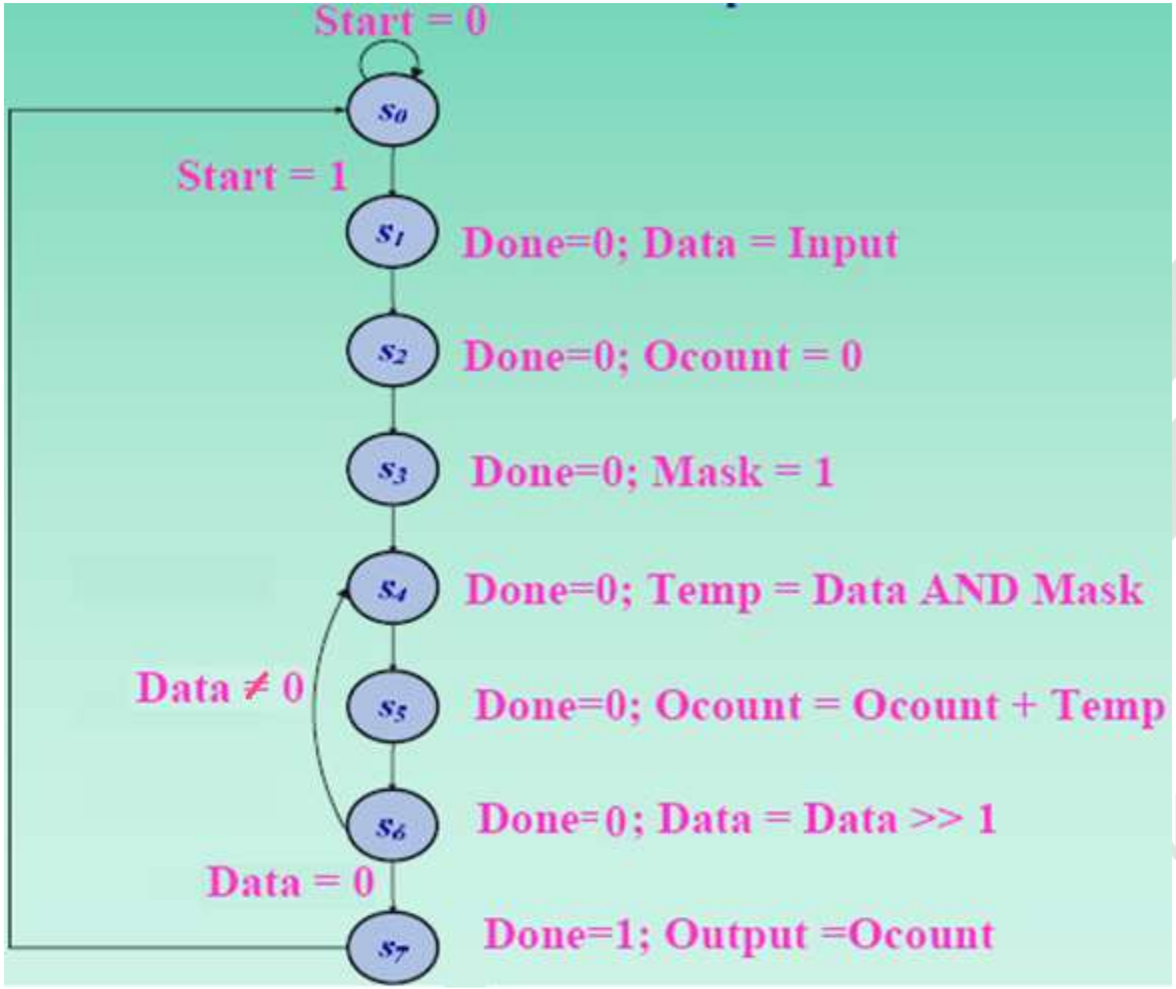
High-level block diagram



Register-transfer-level block diagram

Ones-counter specification





FSDM Definition

- In chapter 1, we defined an FSM as a quintuple ***<S, I, O, f, h>***
  - where ***S*** is a set of states, ***I*** and ***O*** are the sets of input and output symbols: ***f*** : *S × I → S (next state)* , and ***h*** : *S × I → O (FSM output)*
  - More precisely,
    - ***I*** = *A1 × A2 ×…Ak* with A_i $(1 \le i \le k)$ is an input signal
    - ***S*** = *Q1 × Q2 ×…Qm* with Q_i $(1 \le i \le m)$ is the flip-flop output
    - ***O*** = *Y1 × Y2 ×…Yn* with Y_i $(1 \le i \le n)$ is an output signal

- To define a **FSMD**, we define a set of variables ***V = V1 × V2 ×…x Vq*** which defines the state of the datapath by defining the values of all variables in each state.

and $\boldsymbol{I = I_C \times I_D}$

$\boldsymbol{O = O_C \times O_D}$

$\boldsymbol{I_C}$: a set of FSM input, $\boldsymbol{I_D}$: a set of datapath input

$\boldsymbol{O_C}$: a set of FSM output, $\boldsymbol{O_D}$: a set of datapath output

- where $I_C = A_1 \times A_2 \times \ldots A_k$ as before and $I_D = B_1 \times B_2 \times \ldots B_p$,
- where $O_C = Y_1 \times Y_2 \times \ldots Y_n$ as before and $O_D = Z_1 \times Z_2 \times \ldots Z_r$.

FSMD Definition

- We can simplify function $f : (S \times V) \times I \rightarrow S \times V$ by separating it into two parts: f_C and f_D.
  - The function f_C defines the next state of the control unit

$$f_C : S \times I_C \times STAT \rightarrow S$$

  - The function f_D defines the values of datapath variables in the next state, $f_D : S \times V \times I_D \rightarrow V$

$$f_D := \{f_{Di} : V \times I_D \rightarrow V : \{ V_j = e_j \mid V_j\ V,\ e_j\ Expr\ (V \times I_D)\}\}$$

- Also, we can do the same thing with: $h : S \times V \times I \rightarrow O$

$$h_C : S \times I_C \times STAT \rightarrow O_C \quad \textit{(external control output)}$$

$$h_D : S \times V \times I_D \rightarrow O_D \quad \textit{(external datapath output)}$$

STAT: set of all status signals --.> relations between variables and experestion of variables

FSMD specification of Ones-counter

1. *Data* := *Inport*
2. *Ocount* := 0
3. *Mask* := 1

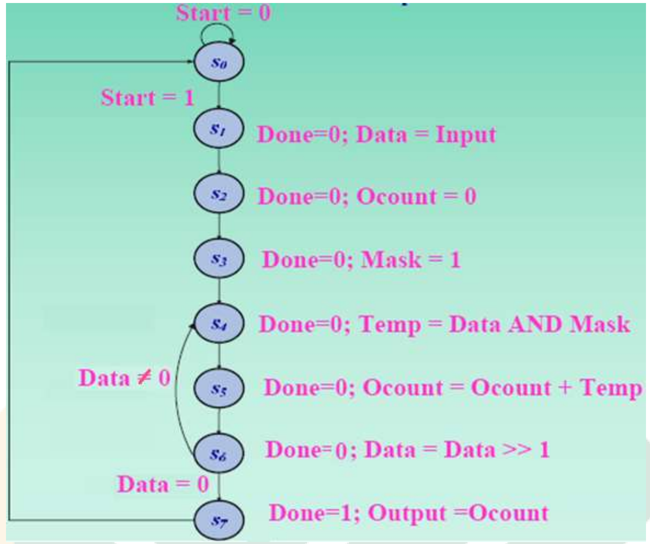
while *Data* ≠ 0 **repeat**

4. *Temp* := *Data* AND *Mask*
5. *Ocount* := *Ocount* + *Temp*
6. *Data* := *Data* >> 1

end while

7. *Outport* := *Ocount*

Basic algorithm for one's counter



PRESENT STATE	NEXT STATE (Start, Data = 0)				CONTROL OUTPUT	DATAPATH OUTPUT	DATAPATH VARIABLES			
	00	01	10	11	Done	Outport	Data	Ocount	Temp	Mask
s_0	s_0	s_0	s_1	s_1	0	Z	X	X	X	X
s_1	s_2	s_2	s_2	s_2	0	Z	*Inport*	X	X	X
s_2	s_3	s_3	s_3	s_3	0	Z	*Data*	0	X	X
s_3	s_4	s_4	s_4	s_4	0	Z	*Data*	*Ocount*	X	1
s_4	s_5	s_5	s_5	s_5	0	Z	*Data*	*Ocount*	*Data* AND *Mask*	*Mask*
s_5	s_6	s_6	s_6	s_6	0	Z	*Data*	*Ocount*+*Temp*	X	*Mask*
s_6	s_4	s_7	s_4	s_7	0	Z	*Data*>>1	*Ocount*	X	*Mask*
s_7	s_0	s_0	s_0	s_0	1	*Ocount*	*Data*	*Ocount*	X	X

State and output table

FSMD specification of Ones-counter (cont.)

1. *Data* := *Inport*
2. *Ocount* := 0
3. *Mask* := 1

while *Data* ≠ 0 **repeat**

4. *Temp* := *Data* AND *Mask*
5. *Ocount* := *Ocount* + *Temp*
6. *Data* := *Data* >> 1

end while

7. *Outport* := *Ocount*

Basic algorithm for one's counter

PRESENT STATE	NEXT STATE (*Start*, *Data* = 0)				CONTROL OUTPUT	DATAPATH OUTPUT	DATAPATH VARIABLES			
	00	01	10	11	*Done*	*Outport*	*Data*	*Ocount*	*Temp*	*Mask*
s_0	s_0	s_0	s_1	s_1	0	Z	X	X	X	X
s_1	s_2	s_2	s_2	s_2	0	Z	*Inport*	X	X	X
s_2	s_3	s_3	s_3	s_3	0	Z	*Data*	0	X	X
s_3	s_4	s_4	s_4	s_4	0	Z	*Data*	*Ocount*	X	1
s_4	s_5	s_5	s_5	s_5	0	Z	*Data*	*Ocount*	*Data* AND *Mask*	*Mask*
s_5	s_6	s_6	s_6	s_6	0	Z	*Data*	*Ocount*+*Temp*	X	*Mask*
s_6	s_4	s_7	s_4	s_7	0	Z	*Data*>>1	*Ocount*	X	*Mask*
s_7	s_0	s_0	s_0	s_0	1	*Ocount*	*Data*	*Ocount*	X	X

State and output table

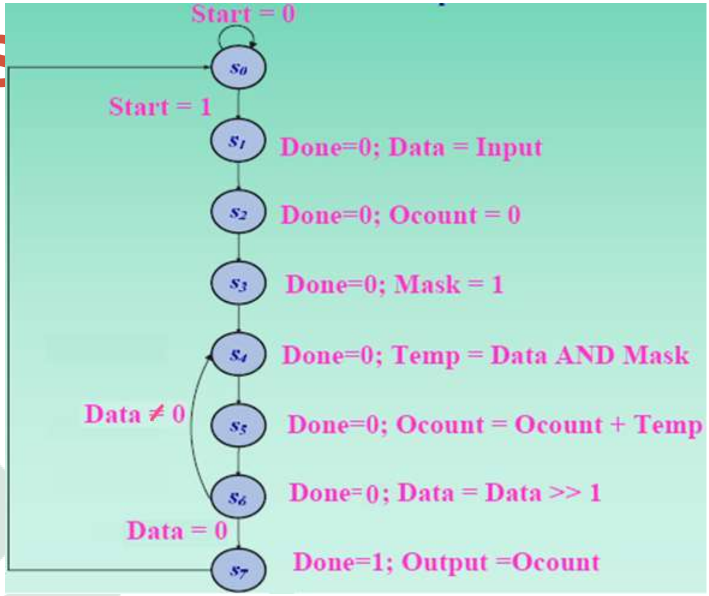
PRESENT STATE	NEXT STATE (*Start*, *Data* = 0)				CONTROL OUTPUT	DATAPATH OUTPUT	DATAPATH VARIABLES
	00	01	10	11	*Done*	*Outport*	
s_0	s_0	s_0	s_1	s_1	0	Z	
s_1	s_2	s_2	s_2	s_2	0	Z	*Data* = *Inport*
s_2	s_3	s_3	s_3	s_3	0	Z	*Ocount* = 0
s_3	s_4	s_4	s_4	s_4	0	Z	*Mask* = 1
s_4	s_5	s_5	s_5	s_5	0	Z	*Temp* = *Data* AND *Mask*
s_5	s_6	s_6	s_6	s_6	0	Z	*Ocount* = *Ocount* + *Temp*
s_6	s_4	s_7	s_4	s_7	0	Z	*Data* = *Data*>>1
s_7	s_0	s_0	s_0	s_0	1	*Ocount*	

State and output table with variable assignments

FSMD specification of Ones

PRESENT STATE	NEXT STATE (Start, Data = 0) 00	01	10	11	CONTROL OUTPUT Done	DATAPATH OUTPUT Outport	DATAPATH VARIABLES
s_0	s_0	s_0	s_1	s_1	0	Z	
s_1	s_2	s_2	s_2	s_2	0	Z	Data = Inport
s_2	s_3	s_3	s_3	s_3	0	Z	Ocount = 0
s_3	s_4	s_4	s_4	s_4	0	Z	Mask = 1
s_4	s_5	s_5	s_5	s_5	0	Z	Temp = Data AND Mask
s_5	s_6	s_6	s_6	s_6	0	Z	Ocount = Ocount + Temp
s_6	s_4	s_7	s_4	s_7	0	Z	Data = Data>>1
s_7	s_0	s_0	s_0	s_0	1	Ocount	

State and output table with variable assignments



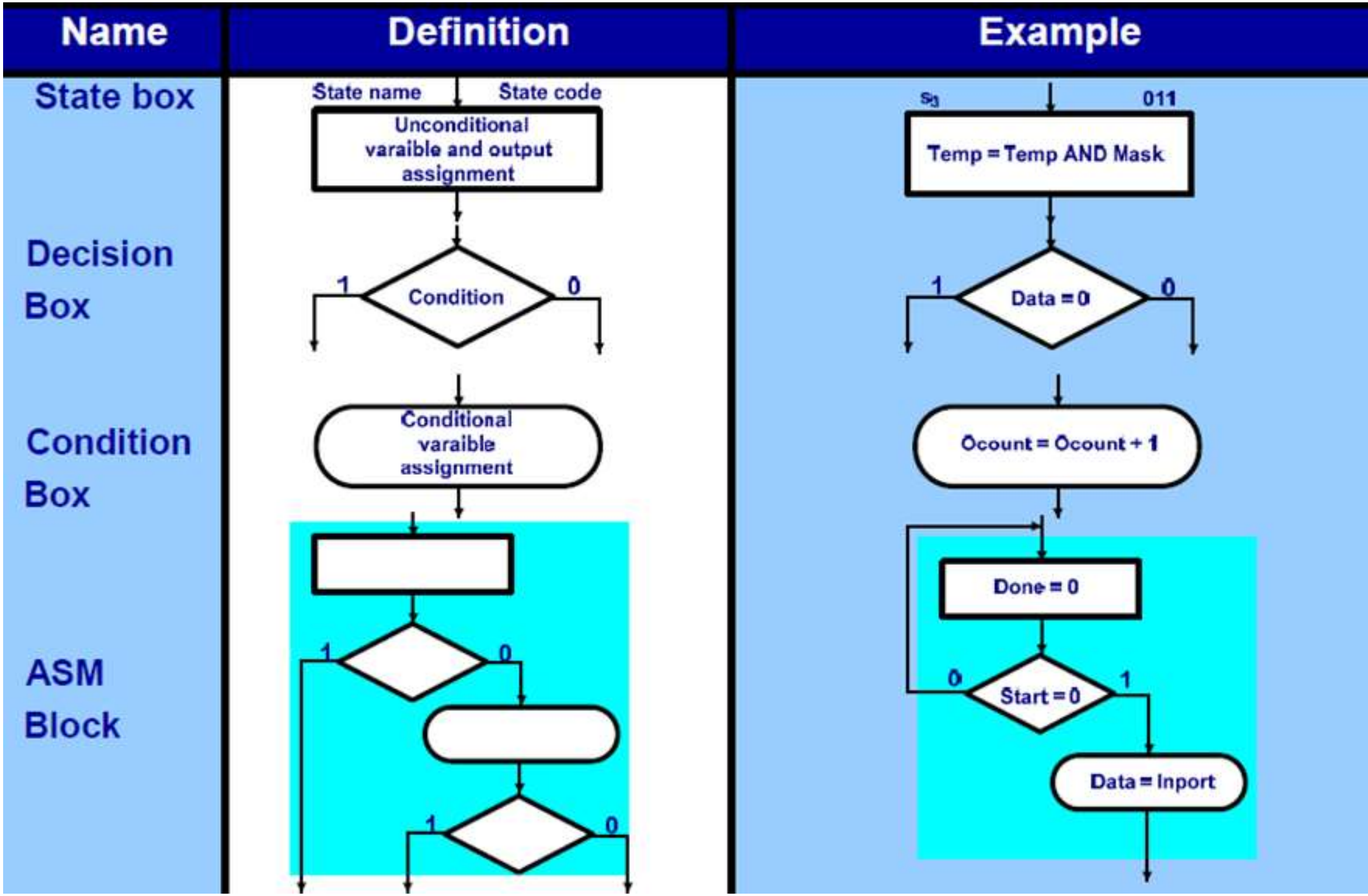
PRESENT STATE	NEXT STATE: CONDITION, STATE	CONTROL AND DATAPATH ACTIONS: CONDITION, ACTIONS
s_0	[Start = 0, s_0; Start = 1, s_1]	[Done = 0; Output = Z]
s_1	s_2	Data = Inport
s_2	s_3	Ocount = 0
s_3	s_4	Mask = 1
s_4	s_5	Temp = Data AND Mask
s_5	s_6	Ocount = Ocount + Temp
s_6	[Data ≠ 0, s_4; Data = 0, s_7]	Data = Data>>1
s_7	s_0	[Done = 1; Output = Ocount]

State-action table

Algorithmic-State-Machine (ASM) chart

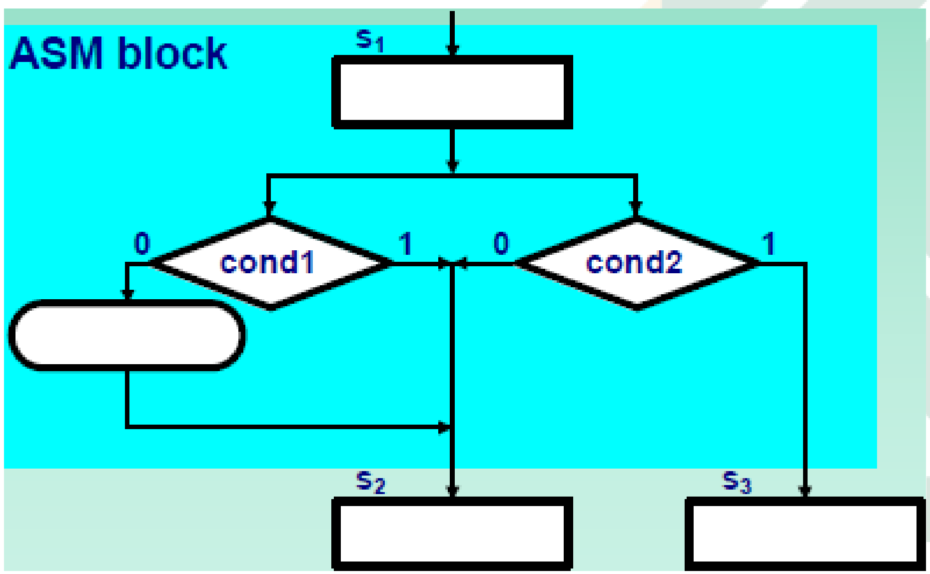
- Graphic representation of FSMD model
- Equivalent to state-action table
- Similar to a flowchart used for program description

ASM Symbols

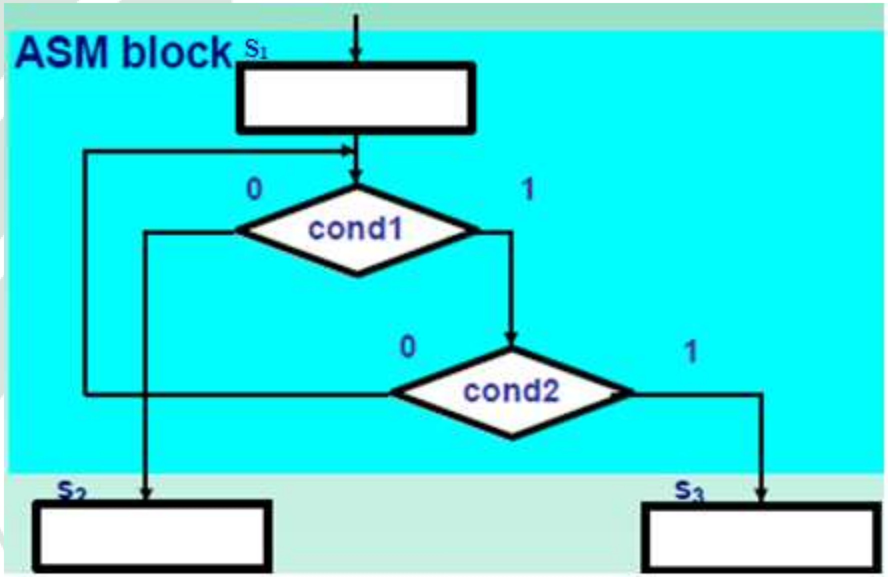

Name
Definition
Example
State box
State name
State code
Unconditional varaible and output assignment
S3
011
Temp = Temp AND Mask
Decision Box
1
Condition
0
1
Data = 0
0
Condition Box
Conditional varaible assignment
Ocount = Ocount + 1
ASM Block
1
0
1
0
Done = 0
0
Start = 0
1
Data = Inport

ASM rules

- **Rule 1**: The chart must define a unique next state for each state and set of conditions.
- **Rule 2**: Every path defined by the network of ~~condition~~ decision boxes must lead to state.

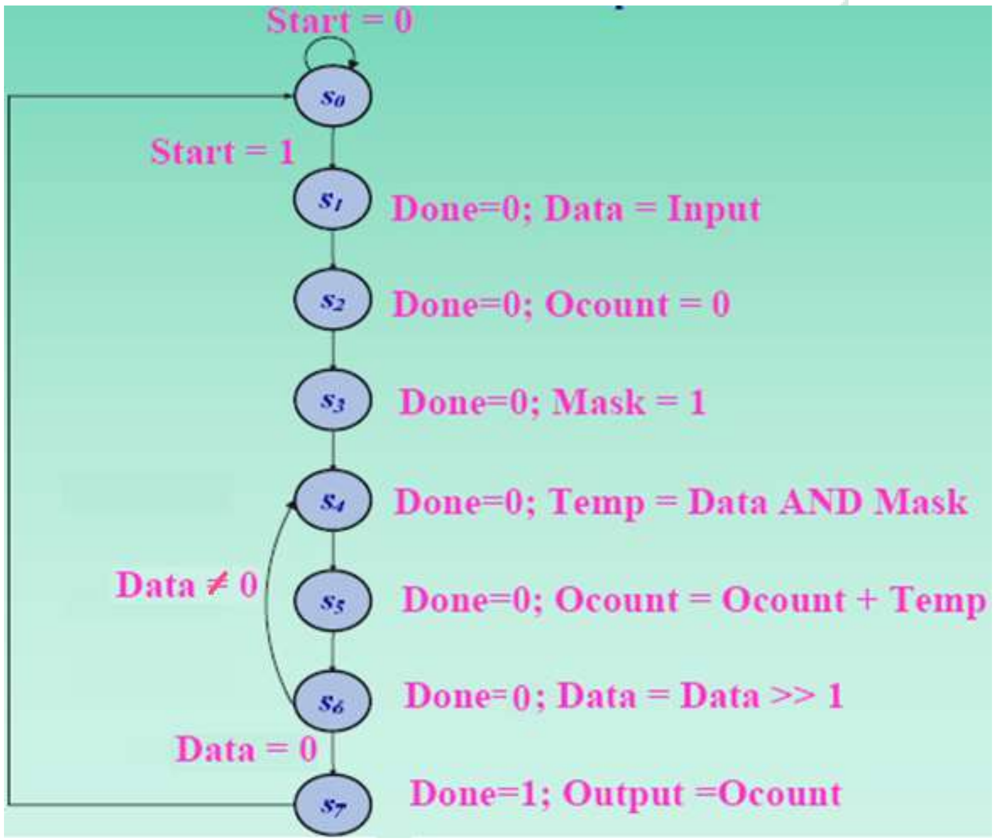


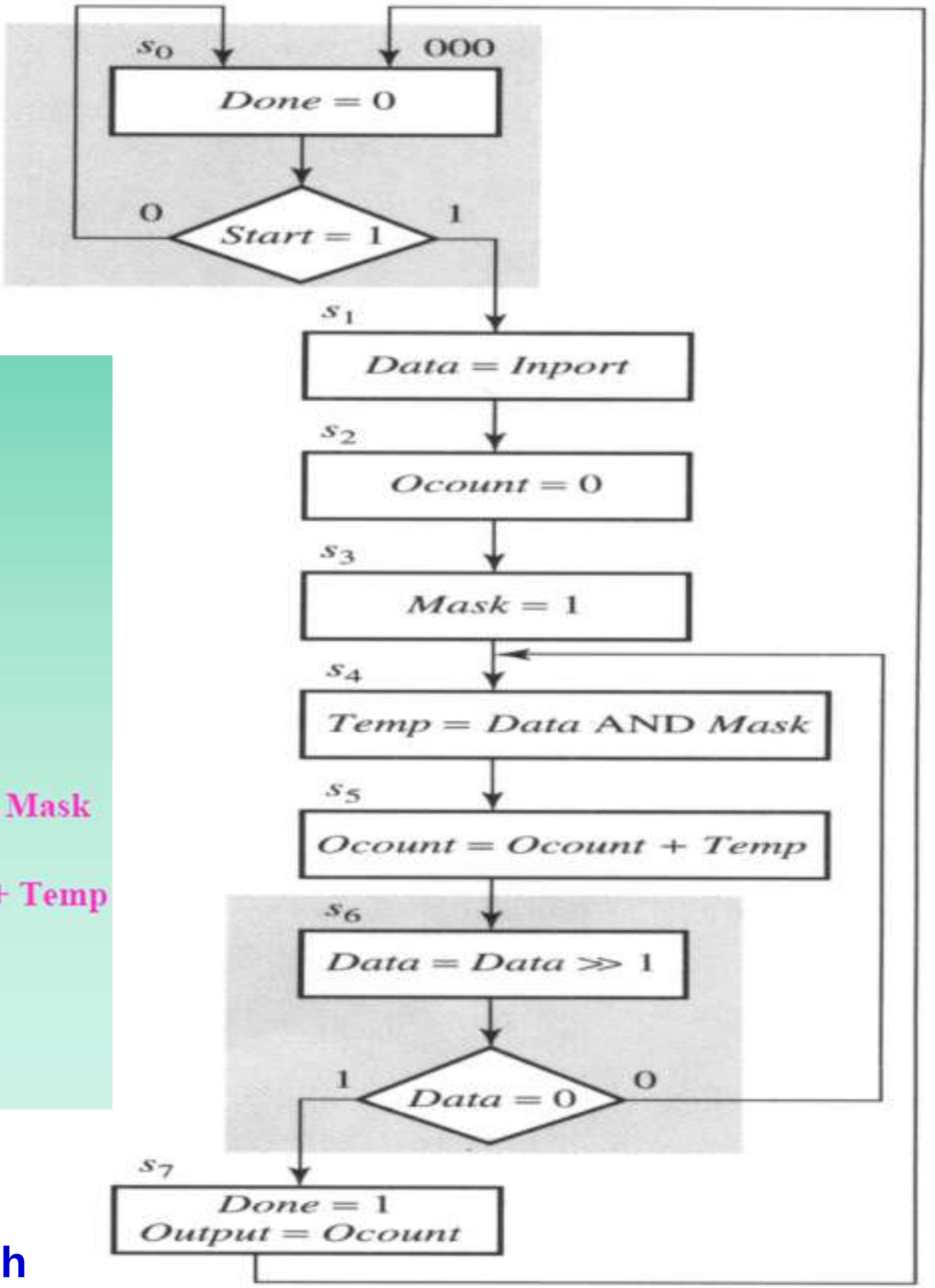
Undefined next state



Undefined exit path

ASM chart for Ones-counter



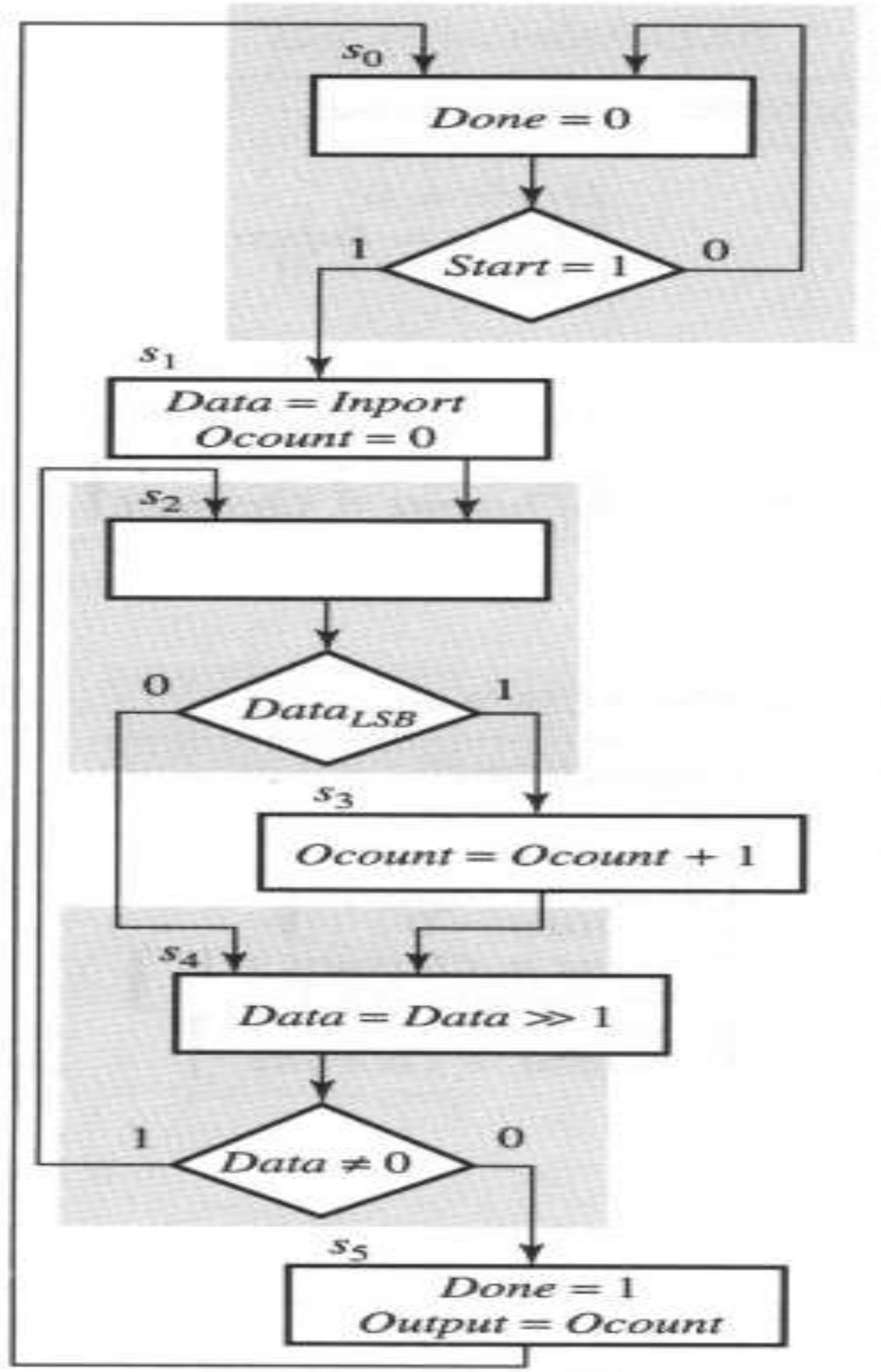


Using for Standard Datapath

ASM chart for Ones-counter

-

Custom Datapath

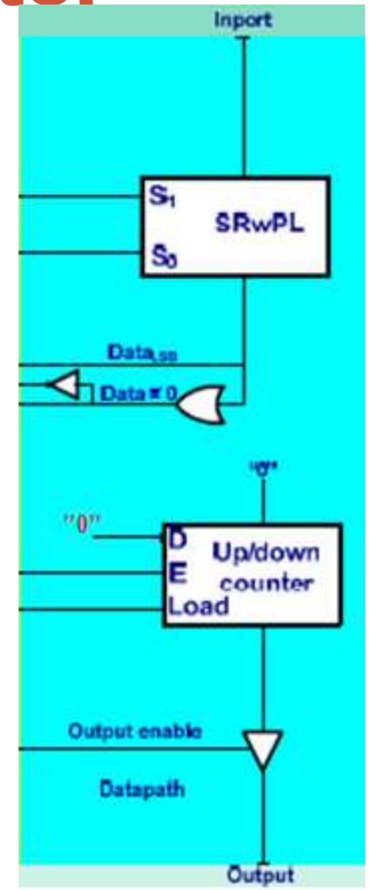


State-based (**Moore**) chart

State-action tables for Ones-counter

PRESENT STATE				NEXT STATE		DATAPATH ACTIONS	
Q_2	Q_1	Q_0	NAME	CONDITION,	STATE	CONDITION,	OPERATIONS
0	0	0	s_0	$Start = 0,$ $Start = 1,$	s_0 s_1		$Done = 0$
0	0	1	s_1		s_2		$Data = Inport$ $Ocount = 0$
0	1	0	s_2	$Data_{LSB} = 1,$ $Data_{LSB} = 0,$	s_3 s_4		
0	1	1	s_3		s_4		$Ocount = Ocount + 1$
1	0	0	s_4	$Data \neq 0,$ $Data = 0,$	s_2 s_5		$Data = Data >> 1$
1	0	1	s_5		s_0		$Done = 1$ $Output = Ocount$

State-based table



$$\mathbf{D_2} = Q_2(\text{next}) = Q_1 Data'_{LSB} + Q_1 Q_0 + Q_2 Q'_0(Data = 0)$$

$$\mathbf{D_1} = Q_1(\text{next}) = Q'_2 Q'_1 Q_0 + Q_1 Q'_0 Data_{LSB} + Q_2 Q'_0(Data \neq 0)$$

$$\mathbf{D_0} = Q_0(\text{next}) = Q'_2 Q'_1 Q'_0 Start + Q_1 Q'_0 Data_{LSB} + Q_2 Q'_0(Data = 0)$$

Logic schematic for Ones-counter

$\mathbf{D_2} = Q_1 Data'_{LSB} + Q_1 Q_0 + Q_2 Q'_0 (Data = 0)$

$\mathbf{D_1} = Q'_2 Q'_1 Q_0 + Q_1 Q'_0 Data_{LSB} + Q_2 Q'_0 (Data \neq 0)$

$\mathbf{D_0} = Q'_2 Q'_1 Q'_0 Start + Q_1 Q'_0 Data_{LSB} + Q_2 Q'_0 (Data = 0)$

$\mathbf{S_1} = s_4 = Q_2 Q'_0$

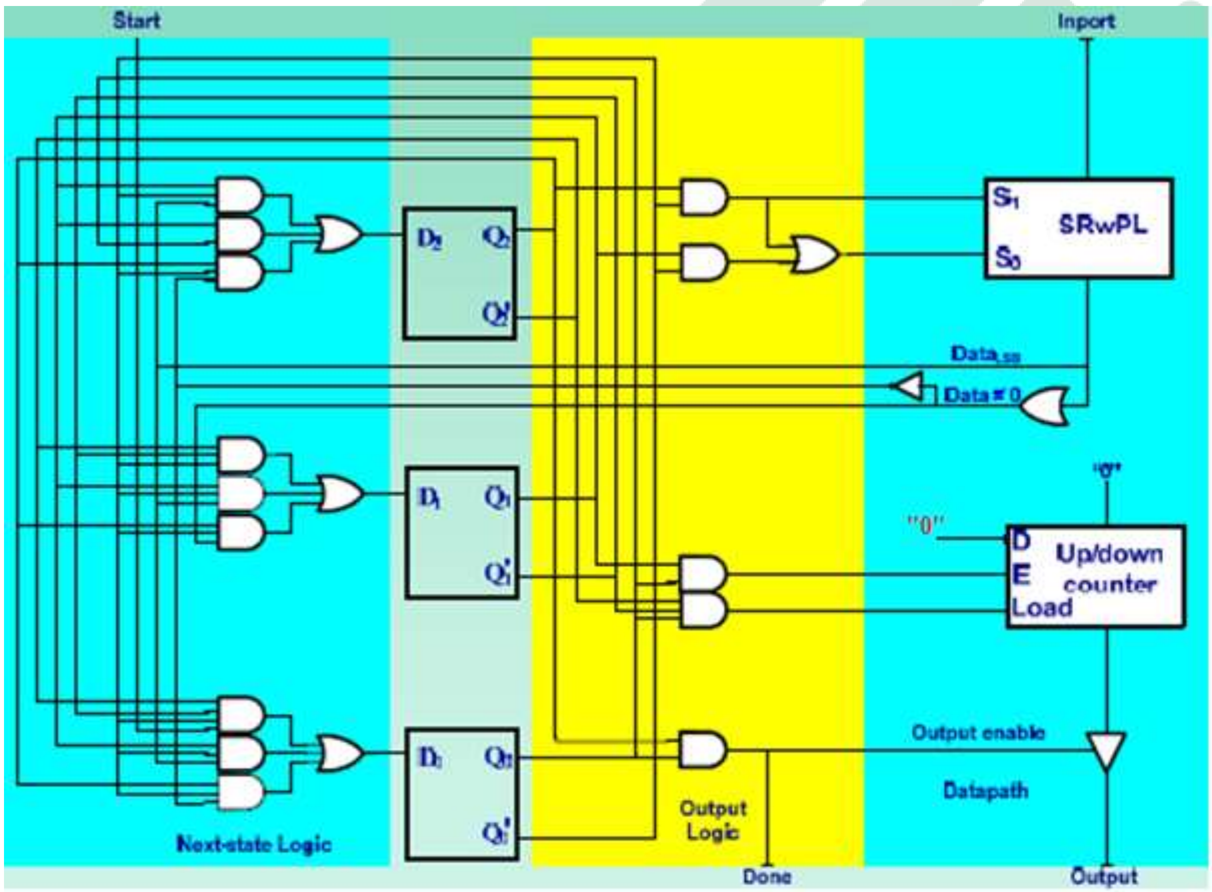
$\mathbf{S_0} = s_1 + s_4 = Q'_2 Q'_1 Q_0 + Q_2 Q'_0$

$\mathbf{E} = s_3 = Q_1 Q_0$

$\mathbf{Load} = s_1 = Q'_2 Q'_1 Q_0$

Done = Output enable

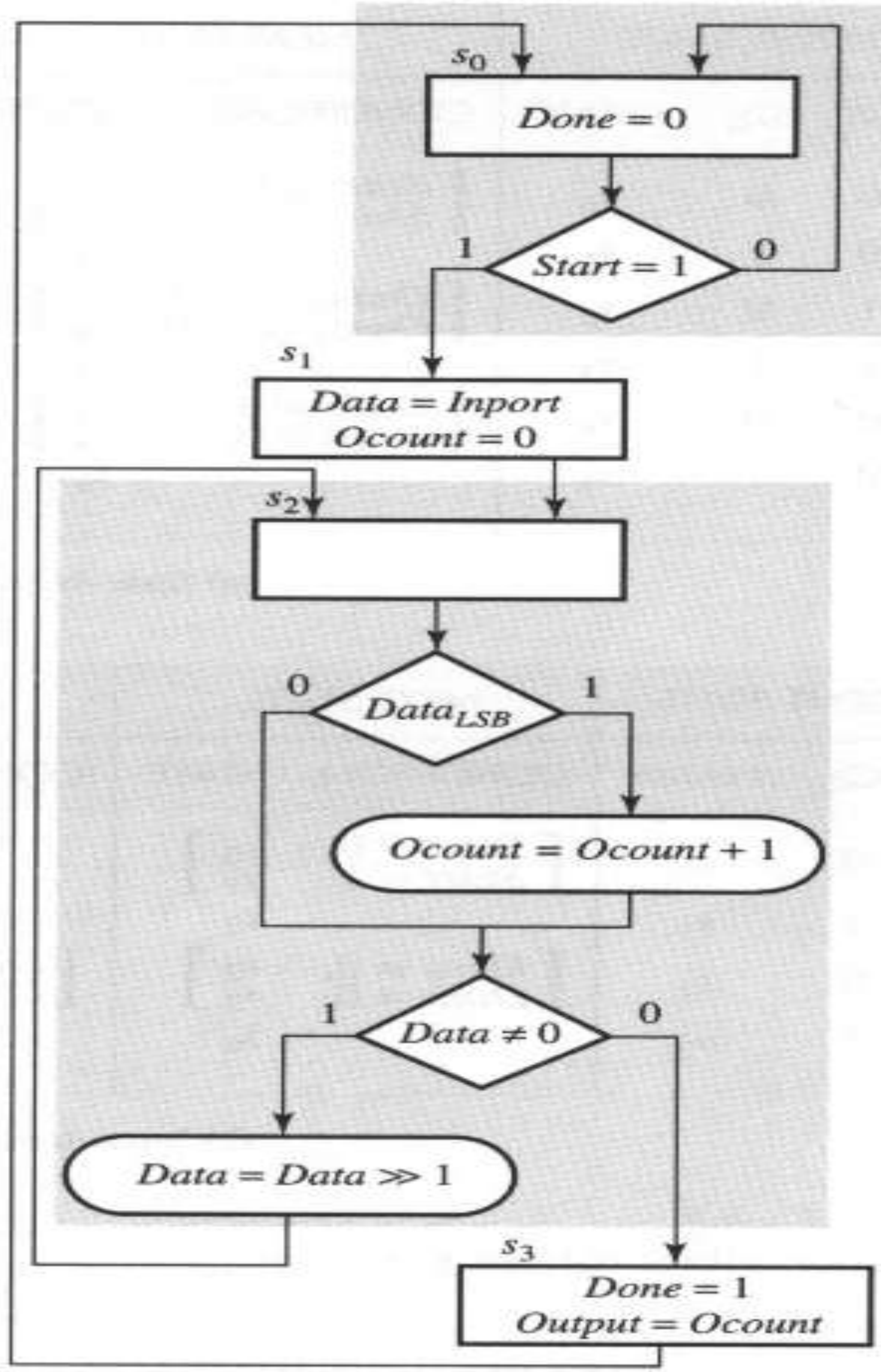
$= s_5 = Q_2 Q_0$



s_i: state name
S_i: control signal of SRwPL

State-based version

ASM chart for Ones-counter
-
Custom Data path (cont.)



Input-based (**Mealy**) chart

State-action tables for Ones-counter

PRESENT STATE			NEXT STATE		DATAPATH ACTIONS	
Q_1	Q_0	NAME	CONDITION,	STATE	CONDITION,	OPERATIONS
0	0	s_0	$Start = 0$, $Start = 1$,	s_0 s_1		$Done = 0$
0	1	s_1		s_2		$Data = Inport$ $Ocount = 0$
1	0	s_2	$Data \neq 0$, $Data = 0$,	s_2 s_3	$Data_{LSB} = 1$, $Data \neq 0$,	$Ocount = Ocount + 1$ $Data = Data >> 1$
1	1	s_3		s_0		$Done = 1$ $Output = Ocount$

Input-based table

$\mathbf{D_1} = Q_1(\text{next}) = s_1 + s_2 = Q'_1Q_0 + Q_1Q'_0$

$\mathbf{D_0} = Q_0(\text{next}) = s_0\text{Start} + s_2(\text{Data} \neq 0)'$

$= Q'_1Q'_0\text{Start} + Q_1Q'_0(\text{Data} \neq 0)'$

Logic schematics for Ones-counter

$\mathbf{D_1} = Q'_1Q_0 + Q_1Q'_0$
$\mathbf{D_0} = Q'_1Q'_0\text{Start} + Q_1Q'_0\,(\text{Data} \neq 0)'$

$\mathbf{S_1} = s_2\,.(\text{Data}\neq 0) = Q_1Q'_0(\text{Data}\neq 0)$
$\mathbf{S_0} = s_1 + s_2(\text{Data}\neq 0) = Q'_1Q_0 + Q_1Q'_0(\text{Data}\neq 0)$
$\mathbf{E} = s_2\text{Data}_{LSB} = Q_1Q'_0\text{Data}_{LSB}$
$\mathbf{Load} = s_1 = Q'_1Q_0$
$\mathbf{Done} = \text{Output enable} = s_3 = Q_1Q_0$

Input-based version